

Logică Matematică și Computațională – SUBIECTE DE EXAMEN

Claudia MUREȘAN

cmuresan@fmi.unibuc.ro, claudia.muresan@g.unibuc.ro, c.muresan@yahoo.com

UNIVERSITATEA DIN BUCUREȘTI, FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

02 SEPTEMBRIE 2024, SERIILE 14 ȘI ID

Timp de lucru: 3 ore.

Materiale permise: orice material tipărit sau scris de mână.

Este interzisă folosirea dispozitivelor electronice.

Este interzisă părăsirea sălii de examen timp de o oră și jumătate din momentul primirii subiectelor.

Punctaj (maxim **11,5 puncte**; nota: $\min\{10, \text{punctaj}\}$; observați că **nota 10** poate fi obținută, în cazul unui punctaj maxim pentru TEMELE COLECTIVE, cu două *cerințe reduse* de programare în Prolog, pentru **jumătate de punctaj**, dacă rezolvările sunt aproape perfecte):

- **1 punct** din oficiu;
- **3 puncte** pentru TEMELE COLECTIVE;
- fiecare dintre cele două cerințe *numerotate* ale fiecărui exercițiu: **1,25 puncte**.

Pentru cerințele de programare în Prolog se poate folosi orice predicat predefinit, precum și orice predicat scris la LABORATOR sau într-o TEMĂ COLECTIVĂ, utilizând directiva pentru includerea bazelor de cunoștințe *labNrlmcVer.pl* și *temeleNr.pl* în cea curentă, cu condiția respectării **denumirilor predicatelor** din FIȘIERELE .PL de la LABORATOR și din ENUNȚURILE TEMELOR COLECTIVE. Toate celelalte predicate auxiliare necesare pentru a defini predicatele cerute trebuie scrise pe lucrarea de examen. Atenție la cerința ca predicatele auxiliare să poată fi aplicate pentru orice argumente de tipurile specificate: mențiunea ca acestea să fie **arbitrare**!

Fiecare coală (nu pagină) din lucrare va fi semnată cu numele și prenumele în clar. Pe prima pagină vor fi scrise și numărul grupei și data examenului.

Toate subiectele vor fi tratate pe lucrarea de examen. Dacă nu aveți nicio idee de abordare a unui exercițiu, atunci veți scrie noțiuni și/sau proprietăți teoretice din curs și/sau predicate din laborator legate de acel exercițiu pe lucrare. Dar **numai** în cazul în care nu știți să abordați exercițiul. Orice încercare de abordare a unui exercițiu valorează mai mult, ca punctaj, decât o astfel de tratare minimală, scriind teorie sau predicate făcute la laborator, iar acestea nu se punctează în plus în cazul în care abordați acel exercițiu.

Exercițiul 1. Să se determine toate funcțiile strict crescătoare $f : \mathcal{L}_2 \times \mathcal{L}_3 \rightarrow \mathcal{L}_4$ de la produsul direct al lanțului cu exact 2 elemente cu lanțul cu exact 3 elemente la lanțul cu exact 4 elemente și să se arate că toate sunt surjective și niciuna dintre acestea nu este morfism de latici:

① matematic;

② prin următoarele predicate în Prolog:

- un predicat unar $fctL2xL3laL4(-ListaFctStrCresc)$, care determină în argumentul său *ListaFctStrCresc* lista funcțiilor strict crescătoare de la $\mathcal{L}_2 \times \mathcal{L}_3$ la $\mathcal{L}_4$, folosind un predicat auxiliar care determină lista funcțiilor strict crescătoare între două poseturi finite arbitrare;
- un predicat zeroar *toatesurj* care întoarce *true* ddacă toate funcțiile strict crescătoare $f : \mathcal{L}_2 \times \mathcal{L}_3 \rightarrow \mathcal{L}_4$ sunt surjective, care apelează predicatul $fctL2xL3laL4(-ListaFctStrCresc)$, apoi aplică listei *ListaFctStrCresc* un predicat care testează dacă toate funcțiile între două mulțimi finite arbitrare dintr-o listă arbitrară de funcții între aceste mulțimi sunt surjective;
- un predicat zeroar *niciunamorf* care întoarce *true* ddacă nicio funcție strict crescătoare $f : \mathcal{L}_2 \times \mathcal{L}_3 \rightarrow \mathcal{L}_4$ nu este morfism de latici, care apelează predicatul $fctL2xL3laL4(-ListaFctStrCresc)$, apoi aplică listei *ListaFctStrCresc* un predicat care testează dacă nicio funcție între două latici finite arbitrare dintr-o listă arbitrară de funcții între aceste latici nu este morfism de latici.

Pentru **jumătate din punctajul** de la **această a doua cerință**, puteți scrie doar predicatul unar $fctL2xL3laL4$ definit ca mai sus.

Exercițiul 2. Fie E mulțimea enunțurilor logicii propoziționale clasice, iar $\alpha, \beta, \varphi \in E$, astfel încât:

$$\varphi = \neg[(\alpha \vee \beta) \rightarrow \neg \alpha].$$

Să se demonstreze că α și φ sunt echivalente semantic: $\alpha \sim \varphi$:

① matematic;

② prin următoarele predicate în Prolog:

- un predicat binar $fi(+Alfa, +Beta)$, care întoarce valoarea de adevăr a enunțului compus φ definit ca mai sus prin conectori logici aplicați la două enunțuri arbitrare α și β în funcție de valorile de adevăr $Alfa$, respectiv $Beta$ ale enunțurilor α , respectiv β într-o interpretare arbitrară;
- un predicat binar $eqalfafi(+Alfa, +Beta)$, care întoarce valoarea de adevăr a enunțului compus $\alpha \leftrightarrow \varphi$, unde φ este definit ca mai sus prin conectori logici aplicați la două enunțuri arbitrare α și β , în funcție de valorile de adevăr $Alfa$, respectiv $Beta$ ale enunțurilor α , respectiv β într-o interpretare arbitrară;
- un predicat zeroar $echivalfafi$, care întoarce $true$ ddacă $\alpha \leftrightarrow \varphi$ este teoremă formală, i.e. ddacă predicatul $eqalfafi(+Alfa, +Beta)$ de mai sus întoarce $true$ pentru toate perechile de valori booleene pentru $(Alfa, Beta)$, determinând valorile lui $eqalfafi(+Alfa, +Beta)$ în toate aceste perechi de valori.

Pentru **jumătate din punctajul** de la **această a doua cerință**, puteți scrie doar predicatul fi .

Desigur, la fel ca în lecțiile de laborator, prin **valoare de adevăr** a unui enunț $\varepsilon \in E$ într-o interpretare $h : V \rightarrow \mathcal{L}_2$ ne referim la valoarea $f(\tilde{h}(\varepsilon))$ a funcției $f \circ \tilde{h} : E \rightarrow \{false, true\}$ în enunțul ε , unde $\tilde{h} : E \rightarrow \mathcal{L}_2$ este prelungirea lui h la mulțimea E a enunțurilor care transformă conectorii logici în operații booleene, iar $f : \mathcal{L}_2 \rightarrow \{false, true\}$ este izomorfismul boolean: $f(0) = false$, $f(1) = true$.

Exercițiul 3. Considerăm signatura de ordinul I: $\tau = (1; 2; 0)$, simbolul de operație unară f , simbolul de relație binară R și simbolul de constantă k , o mulțime $A = \{a, b, c\}$ având $|A| = 3$ și o structură de ordinul I de semnătură τ : $\mathcal{A} = (A, f^A, R^A, k^A)$, cu mulțimea suport A , iar $f^A : A \rightarrow A$ și $R^A \subseteq A^2$, astfel încât, dacă $(A, \leq)$ este posetul având relația de succesiune $\prec = \{(a, b)\}$:

- închiderea reflexivă a relației binare funcționale totale $f^A \subseteq A \times A$ este relația de ordine a posetului descris mai sus: $\mathcal{R}(f^A) = \leq$;
- R^A este relația de echivalență pe A generată de f : $R^A = \mathcal{E}(f^A)$;
- k^A este elementul lui A a cărui clasă de echivalență modulo R^A este singleton: $k^A \in A$, având $|k^A/R^A| = 1$.

Considerăm două variabile distincte $x, y \in Var$ și enunțul:

$$\varepsilon = \forall x \forall y [(f(k)=f(f(x)) \wedge f(x)=f(f(y))) \rightarrow R(f(x), f(y))].$$

Să se determine funcția f^A , relația binară R^A și constanta k^A , apoi să se determine dacă $\mathcal{A} \models \varepsilon$:

- ① matematic;
- ② prin următoarele predicate în Prolog, în care relația de succesiune a posetului $(A, \leq)$ va fi introdusă direct, ca listă de perechi de constante, alături de lista de constante care reprezintă mulțimea A , iar restul argumentelor vor fi calculate în aceste predicate:

- un predicat unar $detf(-Fctf)$, care întoarce în argumentul său $Fctf$ funcția de la A la A care, ca relație binară pe A , are închiderea reflexivă egală cu $\leq$, utilizând un predicat auxiliar care determină relația de ordine a unui poset pe baza mulțimii sale suport și a relației sale de succesiune, precum și un predicat auxiliar $detrelfcttot(-Fctf, +P, +OrdP)$ care primește ca input un poset $(P, OrdP)$ în ultimele sale două argumente și determină, în primul său argument, fiecare relație binară funcțională totală $Fctf$ pe mulțimea P a cărei închidere reflexivă este egală cu $OrdP$;
- un predicat unar $detR(-RelR)$, care calculează în argumentul său $RelR$ relația de echivalență generată de relația binară $Fctf$ determinată de predicatul $detf(-Fctf)$;
- un predicat unar $detk(-Ctk)$, care determină în argumentul său Ctk elementul lui A a cărui clasă modulo $RelR$ are cardinalul 1, unde $RelR$ este relația de echivalență determinată de predicatul $detR(-RelR)$, folosind un predicat auxiliar $detclssgl(-ListElem, +Mult, +Eq)$ care, atunci când primește ca input o mulțime (i.e. listă fără duplicate) $Mult$ și o relație de echivalență Eq pe $Mult$, întoarce în argumentul $ListElem$ lista elementelor lui $Mult$ având clase singleton modulo Eq ;
- un predicat zeroar $verifAsatepsilon$, care întoarce $true$ ddacă $\mathcal{A} \models \varepsilon$, efectuând o demonstrație semantică, prin testarea perechilor de valori din mulțimea A pentru variabilele x, y într-o interpretare arbitrară (**indicație:** atenție la reprezentarea unei funcții ca relație binară funcțională totală, deci ca listă de perechi, care poate fi furnizată unui predicat unar într-o singură clauză, versus reprezentarea ei prin atâtea clauze pentru un predicat binar câte elemente are domeniul acelei funcții; a doua reprezentare e suficientă pentru cerința redusă de mai jos, dar predicatele pentru generare și colectare de funcții scrise la laborator folosesc prima dintre aceste reprezentări).

Pentru **jumătate din punctajul** de la **această a doua cerință**, puteți scrie doar predicatul zeroar $verifAsatepsilon$, introducând direct operația unară f^A , relația binară R^A și constanta k^A în baza de cunoștințe.